

Streams

Java Advanced Developer

StayAhead Training; Sep, 2025

Collection Transformation

Streams

- To derive a new collection/value based on an existing collection
- For example: movies ---> titles of comedies released pre 2000
- Stream: a device for performing a transformation
- Streams are declarative and allow for compiler optimisations
- Sometimes the derived collection is reduced to an aggregate

Map-Filter-Reduce

Streams

- A traditional, imperative transformation:

```
Collection<Integer> nums = List.of(1, 2, 3, 4, 5);
```

```
int sumOfOddCubes = 0;
```

```
for (var num : nums) {  
    if (num % 2 == 1) {  
        var cube = num * num * num;  
        sumOfOddCubes += cube;  
    }  
}
```

FILTER

MAP

REDUCE

Map-Filter-Reduce

Streams

- A declarative SQL transformation:

```
SELECT sum(number)
FROM numbers
WHERE (number % 2) <> 0;
```

- Streams represent an attempt to transform collections declaratively

Map-Filter-Reduce

Streams

- In most cases you should **filter first**
- Expensive mapping of elements that will be discarded is a waste of time
- Map first *only* if the filter logic depends on the mapped value
- The name comes from the big data algorithm - MapReduce

Map-Filter-Reduce

Streams

- The `map` operation is modelled by a `Function`, e.g.:

```
Function<Integer, Integer> toCube = n -> n * n * n;
```

```
// no boxing/unboxing
```

```
IntUnaryOperator toCube = n -> n * n * n;
```


Map-Filter-Reduce

Streams

- The `filter` operation is modelled by a `Predicate`, e.g.:

```
Predicate<Integer> isOdd = n -> n % 2 == 1;
```

Map-Filter-Reduce

Streams

- The `reduce` operation is modelled by a `BinaryOperator`^{*}, e.g.:

```
BinaryOperator<Integer> sum = (total, n) -> total + n;
```
- The first argument is the aggregate; it retains its value between calls
- The second argument is the next element in the stream
- ^{*}A `BinaryOperator` is a `BiFunction` in which all params are the same type

The Stream Pattern

Streams

```
Collection<Integer> nums = List.of(1, 2, 3, 4, 5);
```

```
int sumOfOddCubes = nums.stream()  
    .filter(isOdd)  
    .map(toCube)  
    .reduce(0, sum);
```

The initial value for the first arg. passed to the BinaryOperator's apply method

The Stream Pattern

Streams

- Internally (a simplification):

```
var reducedNum = 0;
for (var num : nums) {
    if (isOdd.test(num)) {
        var mappedNum = toCube.apply(num);
        reducedNum = sum.apply(reducedNum, mappedNum);
    }
}
return reducedNum;
```

The Stream Pattern

Streams

```
Collection<Integer> nums = List.of(1, 2, 3, 4, 5);
```

```
int sumOfOddCubes = nums.stream()  
    .filter(n -> n % 2 == 1)  
    .map(n -> n * n * n)  
    .reduce(0, (total, n) -> total + n);
```


The Stream Pattern

Streams

- The `stream` method returns a `java.util.stream.Stream`:

```
Stream<Integer> s1 = nums.stream();
```

- Intermediate operation methods (e.g. `filter`, `map`) return a new `Stream`:

```
Stream<Integer> s2 = s1.filter(n -> n % 2 == 1);
```

- Terminal operation methods (e.g. `reduce`) return a value/new `Collection`

The Stream Pattern

Streams

- Why a new class (Stream)?
- Why no map, filter, or reduce Collection methods?
- Each intermediate op. method would have to return a new Collection!
- A Stream does not store anything
- A Stream does nothing until a terminal operation method is called

Reducing to a Collection

Streams

- The `collect` method is used to reduce a `Stream` to a new `Collection`
- It accepts a `Supplier` and two `BiConsumers`
- The `Supplier` supplies the new `Collection`
- The 1st `BiConsumer` (accumulator) adds each element to the new `Collection`
- The 2nd `BiConsumer` (combiner) combines n new `Collections` into one and is used when processing a `Stream` in parallel (two or more threads)

Reducing to a Collection

Streams

```
Supplier<ArrayList<Integer>>  
    newList = () -> new ArrayList<Integer>();
```

```
BiConsumer<ArrayList<Integer>, Integer>  
    addToList = (list, num) -> list.add(num);
```

```
BiConsumer<ArrayList<Integer>, ArrayList<Integer>>  
    combineLists = (list1, list2) -> list1.addAll(list2);
```

Reducing to a Collection

Streams

```
Supplier<ArrayList<Integer>>  
    newList = ArrayList::new;
```

```
BiConsumer<ArrayList<Integer>, Integer>  
    addToList = ArrayList::add;
```

```
BiConsumer<ArrayList<Integer>, ArrayList<Integer>>  
    combineLists = ArrayList::addAll;
```


Reducing to a Collection

Streams

```
Collection<Integer> nums = List.of(1, 2, 3, 4, 5);
```

```
ArrayList<Integer> oddCubes = nums.stream()  
    .filter(isOdd)  
    .map(toCube)  
    .collect(newList, addToList, combineLists);
```

Reducing to a Collection

Streams

- Internally (a simplification):

```
ArrayList<Integer> newNums = newList.get();  
for (var num : nums) {  
    if (isOdd.test(num)) {  
        var mappedNum = toCube.apply(num);  
        addToList.accept(newNums, mappedNum);  
    }  
}  
return newNums;
```

Reducing to a Collection

Streams

```
Collection<Integer> nums = List.of(1, 2, 3, 4, 5);
```

```
ArrayList<Integer> oddCubes = nums.stream()  
    .filter(n -> n % 2 == 1)  
    .map(n -> n * n * n)  
    .collect(  
        ArrayList::new,  
        ArrayList::add,  
        ArrayList::addAll  
    );
```


Primitive Streams

Streams

- Avoid boxing & unboxing with `IntStream`, `LongStream`, & `DoubleStream`
- Additional terminal operation methods:
 - `sum`
 - `min`
 - `max`
 - `average`
 - `summaryStatistics`

Primitive Streams

Streams

```
Collection<Integer> nums = List.of(1, 2, 3, 4, 5);
```

```
int sumOfOddCubes = nums.stream()  
    .filter(n -> n % 2 == 1)  
    .mapToInt(n -> n)  
    .map(n -> n * n * n)  
    .sum();
```

Returns an IntStream

Optional Streams

- The sum terminal operation has an identity - 0
- It will produce a value even if the stream on which it is called is empty
- Other terminal operations do not have an identity, e.g. min, max, average
- Such operation methods return an Optional
- Optional is a wrapper for an object that may be empty

Optional Streams

// note that nums may be an empty list!

```
OptionalInt optionalMin = nums  
    .stream()  
    .mapToInt(n -> n)  
    .min();
```

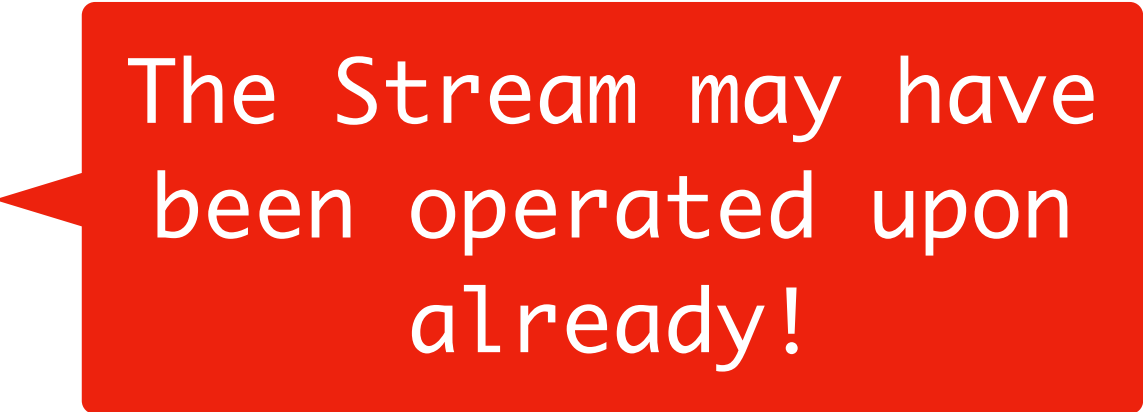
Optional has other methods
including `orElse` and `orElseThrow`
to avoid coding if statements

```
if (optionalMin.isPresent()) {  
    System.out.println(optionalMin.getAsInt());  
} else {  
    System.out.println("No minimum");  
}
```

Good Practice

Streams

- Call only **one method** on a Stream, even if it's an intermediate operation
- Don't assign Streams to variables
- Don't code methods with Stream parameters
- A Stream should **not modify** the Collection upon which it's operating
- A Stream should not have side-effects



The Stream may have been operated upon already!

Creating Streams

Streams

```
Collection<Integer> nums = List.of(1, 2, 3);  
Stream<Integer> fromCollection = nums.stream();
```

```
int[] nums = new int[] {1, 2, 3};  
Stream<Integer> fromArray = Arrays.stream(nums);
```

```
Stream<Integer> fromVarargs = Stream.of(1, 2, 3);
```

```
Stream<Integer> empty = Stream.empty();
```

Creating Streams

Streams

```
// typically used with the limit intermediate operation  
Supplier<Integer> supplier = () -> 42;  
Stream<Integer> fromSupplier = Stream.generate(supplier);
```

```
UnaryOperator<Integer> function = n -> n + 1;  
Stream<Integer> fromFunction = Stream.iterate(0, function);
```

```
Stream<Integer> s1 = Stream.of(1, 2, 3);  
Stream<Integer> s2 = Stream.of(4, 5, 6);  
Stream<Integer> s3 = Stream.concat(s1, s2);
```

Seed; initial value
of argument passed
to the Function

Creating Streams

Streams

// 1st arg. is from incl.; 2nd is to excl.

```
IntStream fromRange = IntStream.range(1, 11);
```

// 1st arg. is num. elements; 2nd is lower bound incl.

// 3rd is upper bound excl.; (also longs and doubles)

```
IntStream fromRandom = new Random().ints(5, 1, 11);
```

// produces a Stream of code points

```
IntStream fromString = "Hello".chars();
```


Creating Streams

Streams

```
Path path = Path.of("somefile.txt");  
Stream<String> fromFiles = Files.lines(path);
```

```
BufferedReader reader = Files.newBufferedReader(path);  
Stream<String> fromReader = reader.lines();
```

```
String csv = "Tom,Dick,Harry";  
Pattern pattern = Pattern.compile(",");  
Stream<String> fromPattern = pattern.splitAsStream(csv);
```

Creating Streams

Streams

```
Stream.Builder<String> streamBuilder = Stream.builder();  
streamBuilder.add("Tom").add("Dick").add("Harry");  
Stream<String> fromBuilder = streamBuilder.build();
```

```
// assuming a HttpClient and HttpRequest  
HttpResponse<Stream<String>> response = httpClient.send(  
    request, HttpRepsonse.BodyHandlers.ofLines());  
Stream<String> fromResponseBody = response.body();
```

Intermediate Operations: map

Streams



Intermediate Operations: map

Streams

- **map** transforms (derives a new object from) each element using a **Function**

```
stringStream.map(str -> str.toLowerCase())
```

```
stringStream.map(String::toLowerCase)
```

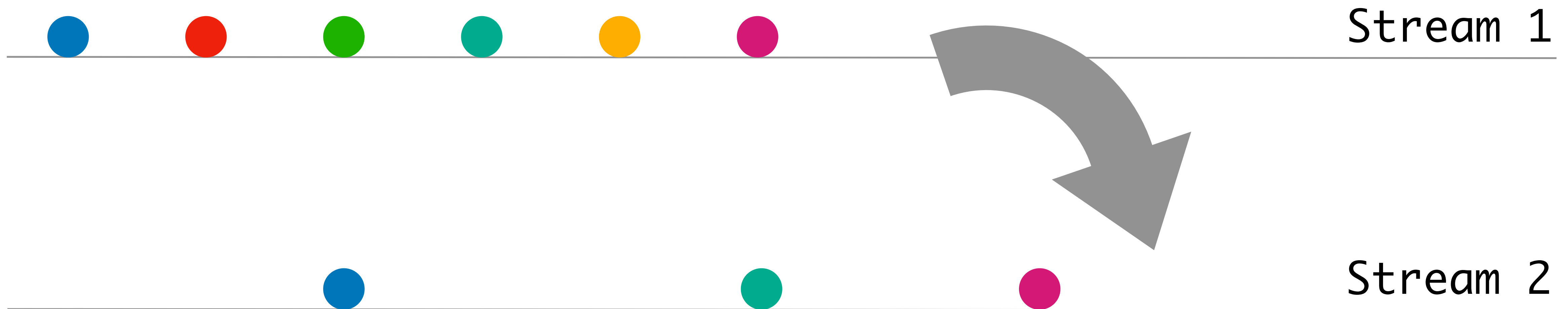
- The derived object type need **not** match the base object type

```
stringStream.map(str -> str.length())
```

```
stringStream.map(String::length)
```

Intermediate Operations: filter

Streams



Intermediate Operations: filter

Streams

- `filter` discards elements using a `Predicate`

```
// discard Strings that don't start with ERROR  
stringStream.filter(str -> str.startsWith("ERROR"))
```

```
// discard empty Strings  
stringStream.filter(str -> !str.isEmpty())
```

Intermediate Operations: flatMap

Streams

- Consider the following classes representing a 1..n relationship:

```
record Player (String name, Position position) {}  
record Team (String name, Set<Player> players) {}
```

- How might one transform a Collection of Teams into a Set of Players?

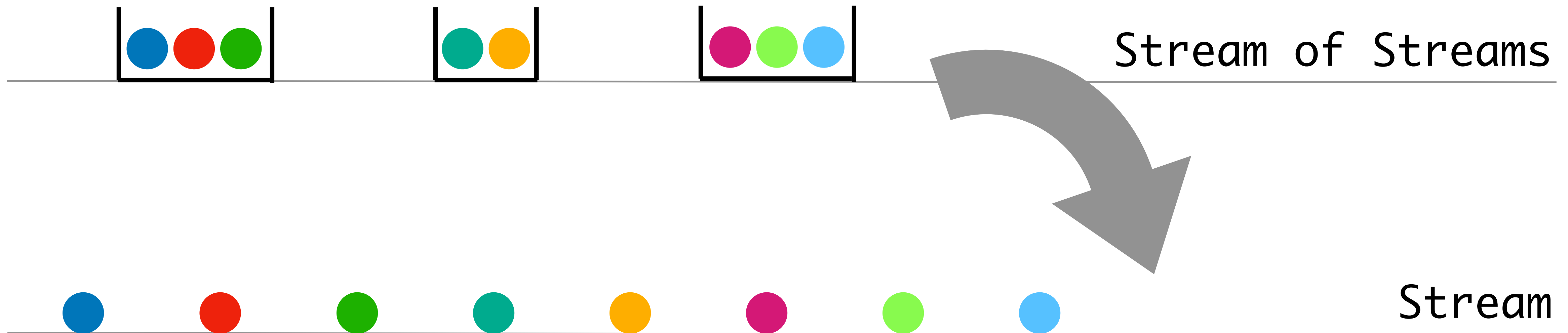
// will produce a Set of Sets!

```
Set<Player> players = teams.stream()  
    .map(Team::players)  
    .collect(HashSet::new, HashSet::add, HashSet::addAll);
```



Intermediate Operations: flatMap

Streams



Intermediate Operations: flatMap

Streams

- `flatMap` flattens a Stream of Streams using a `Function`

// note Function method's return value is a Stream

```
Set<Player> players = teams.stream()  
    .flatMap(team -> team.players().stream())  
    .collect(HashSet::new, HashSet::add, HashSet::addAll);
```

Intermediate Operations: flatMap

Streams

- Internally (a simplification):

```
Set<Player> players = supplier.get();  
for (var team : teams) {  
    for (var player : team.players()) {  
        accumulator.accept(players, player);  
    }  
}  
return players;
```

Intermediate Operations: mapMulti

Streams

- `mapMulti` is like `flatMap` but provides for more control over the mapping
- It accepts a `BiConsumer<? super T, ? super Consumer<R>>`

```
var strings = List.of("1", "two", "3", " 4", "5");  
var numbers = strings.stream()  
    .<Integer>mapMulti((str, consumer) -> {  
        try {  
            consumer.accept(Integer.parseInt(str));  
        } catch (NumberFormatException ignore) {}  
    }).toList();
```

If an exception is thrown the Consumer is not called/the element is skipped

Intermediate Operations: distinct

Streams

- `distinct` removes duplicates (using hashCode and equals internally)

```
// adds elements to a Set internally  
stream.distinct()
```

Intermediate Operations: sorted

Streams

- `sorted` sorts the elements

```
// assuming comparable elements  
stream.sorted()
```

```
// sorted may be passed a Comparator  
record Thing(String name, int size) {}  
thingStream.sorted(Comparator.comparing(Thing::name))
```


Intermediate Operations: skip & limit

Streams

- **skip** skips/ignores the given number of elements

```
stream.skip(1)
```

- **limit** dictates the number of elements

```
stream.limit(5)
```

```
var nums = Stream.iterate(0, n -> n + 1)  
    .skip(2).limit(5).toList();  
// nums ==> [2, 3, 4, 5, 6]
```

Intermediate Operations: dropWhile

Streams

- `dropWhile` drops elements until the `Predicate`'s test yields `false`
`stringStream.dropWhile(str -> str.startsWith("ERROR"))`
- Differs from `filter` insofar as once the gate is opened it remains open

Intermediate Operations: takeWhile

Streams

- `takeWhile` accepts elements until the `Predicate`'s test yields `false`
`stringStream.takeWhile(str -> str.startsWith("ERROR"))`
- Differs from `filter` insofar as once the gate is shut it remains shut

Intermediate Operations: peek

Streams

- `peek` allows for debugging the pipeline using a `Consumer`
`stream.peek(System.out::println)`

Terminal Operations

Streams

- `reduce` aggregates the elements into a single value (see slide 8)
- `collect` places each element into a new `Collection` (see slide 14)
- Primitive Streams have aggregating methods such as `sum`, `min`, and `max`

Terminal Operations: count

Streams

- `count` returns the number of elements in the stream

```
var nums = List.of(5, 11, 4, 7, 2, 18, 10, 3, 17, 9);
```

```
long numberOfOddNums = nums  
    .stream()  
    .filter(n -> n % 2 == 1)  
    .count();
```

```
// numberOfOddNums ==> 6
```


Terminal Operations: `forEach`

Streams

- `forEach` processes each element using a `Consumer`

```
var names = List.of("John", "Paul", "George", "Ringo");
```

```
names  
    .stream()  
    .map(String::toLowerCase)  
    .forEach(System.out::println);
```

Terminal Operations: min & max

Streams

- `min` returns the smallest and `max` the biggest element using a `Comparator`

```
var names = List.of("John", "Paul", "George", "Ringo");
```

```
Optional<String> smallest = names.stream()  
    .min((s1, s2) -> s1.compareTo(s2));  
// smallest ==> Optional[George]
```

```
Optional<String> biggest = names.stream()  
    .max(String::compareTo);  
// biggest ==> Optional[Ringo]
```


Terminal Operations: findFirst

Streams

- `findFirst` returns the first element (assuming there is one)

```
var names = List.of("John", "Paul", "George", "Ringo");
```

```
Optional<String> first = names.stream().findFirst();  
// first ==> Optional[John]
```

- NB: if the stream is unordered `findFirst` returns any element!

Terminal Operations: findAny

Streams

- `findAny` returns any element (assuming there is one or more)

```
var names = List.of("John", "Paul", "George", "Ringo");
```

```
Optional<String> any = names.stream().findAny();  
// any ==> Optional[?]
```

- `findAny` will likely return the first element unless used in parallel

Terminal Operations: anyMatch

Streams

- `anyMatch` returns `true` if any element is a match for the given `Predicate`

```
var names = List.of("John", "Paul", "George", "Ringo");
```

```
boolean hasNameStartingWithG = names  
    .stream()  
    .anyMatch(name -> name.startsWith("G"));
```

```
// hasNameStartingWithG ==> true
```


Terminal Operations: allMatch

Streams

- `allMatch` returns `true` if every element is a match for the given `Predicate`

```
var names = List.of("John", "Paul", "George", "Ringo");
```

```
boolean allNamesFourCharsLong = names  
    .stream()  
    .allMatch(name -> name.length() == 4);
```

```
// allNamesFourCharsLong ==> false
```

Terminal Operations: noneMatch

Streams

- `noneMatch` returns `true` if no element is a match for the given `Predicate`

```
var names = List.of("John", "Paul", "George", "Ringo");
```

```
boolean noVeryShortNames = names  
    .stream()  
    .noneMatch(name -> name.length() <= 3);
```

```
// noVeryShortNames ==> true
```

Terminal Operations: collect

Streams

- The `collect` method is overloaded to accept a `Collector`
- A `Collector` is an object that performs a reduction operation
- Each static method in the `Collectors` class returns a `Collector`
- Each `Collector` serves to simplify a common use case

Collectors: toList

Streams

```
import static java.util.stream.Collectors.*;  
var nums = List.of(1, 2, 3, 4, 5);
```

```
ArrayList<Integer> list = nums.stream().collect(toList());
```

```
// to avoid expensive copying of arrays internally  
// or to collect into a Collection of your choosing
```

```
ArrayList<Integer> list = nums.stream().collect(  
    toCollection() -> new ArrayList<>(100_000));
```

Collectors: toList

Streams

```
import static java.util.stream.Collectors.*;  
var nums = List.of(1, 2, 3, 4, 5);
```

```
// returns an immutable List  
List<Integer> list = nums.stream()  
    .collect(toUnmodifiableList());
```

```
// no collect - returns an immutable list...  
// with no copying of arrays internally  
List<Integer> list = nums.stream().toList();
```


Collectors: toSet

Streams

```
import static java.util.stream.Collectors.*;  
var nums = List.of(1, 1, 2, 2, 3, 3);
```

```
Set<Integer> set = nums.stream().collect(toSet());
```

```
// returns an immutable Set
```

```
Set<Integer> set = nums.stream().collect(toUnmodifiableSet());
```

Collectors: toArray

Streams

```
var nums = List.of(1, 2, 3, 4, 5);
```

```
Object[] array = nums.stream().toArray();
```

```
Integer[] array = nums.stream().toArray(Integer[]::new);
```

Collectors: joining

Streams

```
import static java.util.stream.Collectors.*;
var strings = List.of("H", "e", "l", "l", "o");

String greeting = strings.stream().collect(joining());
// greeting ==> Hello

String csv = strings.stream().collect(joining(","));
// csv ==> H,e,l,l,o
```


Collectors: joining

Streams

```
import static java.util.stream.Collectors.*;  
var strings = List.of("H", "e", "l", "l", "o");
```

```
String jsonArray = strings  
    .stream()  
    .map(str -> String.format("\"%s\"", str))  
    .collect(joining(", ", "[", "]"));
```

```
// jsonArray ==> ["H", "e", "l", "l", "o"]
```

Collectors: toMap

Streams

```
import static java.util.stream.Collectors.*;
record User(long id, String name, String email) {}
var users = List.of(
    new User(1L, "Tom", "tom@mail.com"),
    new User(2L, "Dick", "dick@mail.com"),
    new User(3L, "Harry", "harry@mail.com"));
```

```
Map<Long, User> map = users.stream()
    .collect(toMap(User::id, user -> user));
```

toMap accepts a
key Function &
value Function

Collectors: partitioningBy

Streams

```
import static java.util.stream.Collectors.*;  
var nums = List.of(15, 3, 21, 7, 88, 47, 4, 50);
```

```
Map<Boolean, List<Integer>> map = nums.stream()  
    .collect(partitioningBy(num -> num < 10));
```

partitioningBy
accepts a Predicate

```
// map ==> {false=[15, 21, 88, 47, 50], true=[3, 7, 4]}
```


Collectors: groupingBy

Streams

```
import static java.util.stream.Collectors.*;
record Player(String name, String team, String position) {}
var players = List.of(
    new Player("Tom", "Reds", "Forward"),
    new Player("Dick", "Blues", "Forward"),
    new Player("Harry", "Reds", "Defender"));
```

```
Map<String, List<Player>> map = players.stream()
    .collect(groupingBy(Player::team));
```

groupingBy
accepts a Function

Collectors: groupingBy

Streams

- The `groupingBy` method may be passed a second argument - a downstream `Collector` to be applied to each `Collection` of values

```
import static java.util.stream.Collectors.*;
```

```
...
```

```
Map<String, Long> map = players.stream()  
    .collect(groupingBy(Player::team, counting()));
```

```
// map ==> {Reds=2, Blues=1}
```


Intermediate Collectors

Streams

- An intermediate Collector is one that filters, maps, or flat maps values after grouping, and must be passed a downstream Collector

```
import static java.util.stream.Collectors.*;
```

```
...
```

```
Map<String, List<String>> map = players.stream()  
    .collect(groupingBy(Player::team,  
        mapping(Player::name, toList())));
```

```
// map ==> {Reds=[Tom, Harry], Blues=[Dick]}  
// see also filtering and flatMapping
```

Terminal Collectors

Streams

- A terminal Collector mirrors the behaviour of terminal operations like `min`, `max`, `sum`, and `average` but is used when grouping

```
import static java.util.stream.Collectors.*;  
record Movie(String title, String genre, int rating) {}  
...
```

```
Map<String, Double> map = movies.stream()  
    .collect(groupingBy(Movie::genre,  
        mapping(m -> m, averagingInt(Movie::rating))));
```

```
// see also minBy, maxBy, summingInt etc.
```